

YT-AGENT-AI - Documentacion del Proyecto

TicSocial S. A. S.

2026

Indice de Contenidos

YT-AGENT-AI - Documentacion del Proyecto	1
Estado Actual del Proyecto	1
Descripcion General	1
Proposito Actual	1
Flujo de Ejecución	1
Stack Tecnologico	2
Estructura de Directorios	2
Componentes Principales	3
1. Servicios	3
2. Interfaz Streamlit	3
3. API REST	3
4. Excepciones Personalizadas	3
5. Base de Datos	3
6. Seguridad: Autenticacion y Throttling	4
Cambios Recientes	4
Configuracion del Entorno	4
Despliegue	4
Mantenimiento	5
Limpieza de Archivos Temporales	5
Preguntas Frecuentes	5
Checklist de Verificacion	5
Historial de Reorganizacion de Ramas Git	5

YT-AGENT-AI - Documentacion del Proyecto

Estado Actual del Proyecto

YT-AGENT-AI es una aplicacion web avanzada que permite analizar videos de YouTube y obtener un **Reporte de Contenido** detallado. Utiliza inteligencia artificial para transcribir el audio y generar un analisis estructurado de lo que se dice en el video.

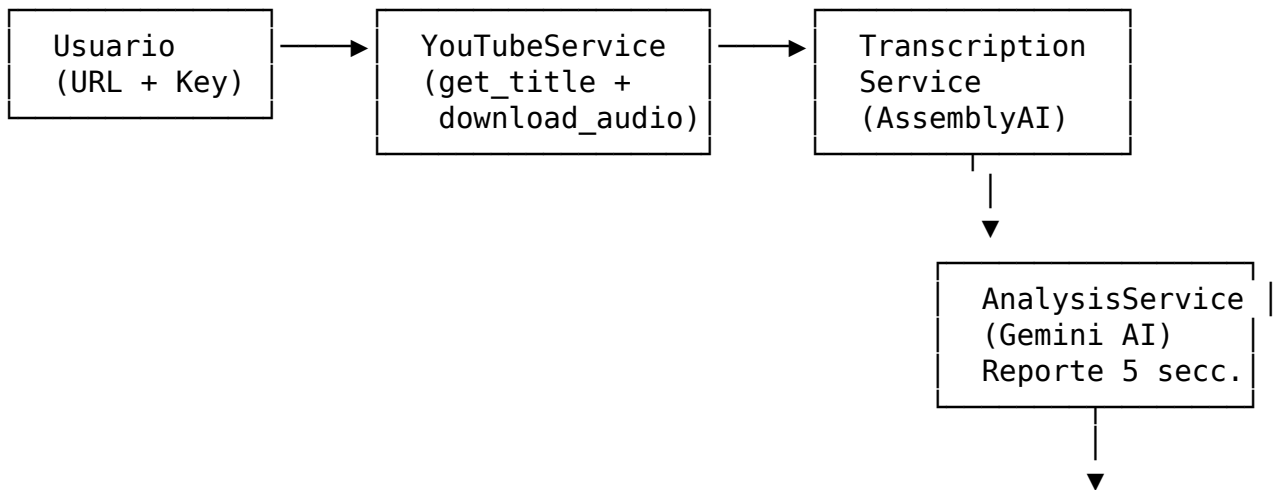
Descripcion General

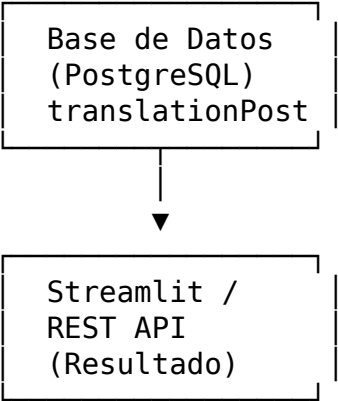
El proyecto fue originalmente diseñ para traducir letras de canciones de YouTube. Tras una evolucion significativa, el proposito del proyecto fue completamente redefinido: en lugar de traducir, ahora se **analiza y reporta** el contenido de cualquier video de YouTube.

Proposito Actual

Ingresar una URL de YouTube → Descargar audio → Transcribir (AssemblyAI) → Generar un **Reporte de Contenido** estructurado usando Google Gemini (5 secciones).

Flujo de Ejecución





Stack Tecnologico

Componente	Tecnologia	Version
Backend Framework	Django	4.1
Frontend	Streamlit	1.33.0
Transcripcion	AssemblyAI	0.36.0
Inteligencia Artificial	Google Gemini (cadena con fallback)	gemini-3.5-flash-lite (default)
Descarga de Video	yt-dlp	2025.9.26
API framework	Django REST Framework	3.14.0
Base de Datos	PostgreSQL	(Neon Cloud)
Contenedorizacion	Docker + Docker Compose	3.8
ORM	Django ORM	-

Estructura de Directorios

```
Backend/
├── app.py                                # Frontend Streamlit (cliente HTTP de la API)
├── .env.example                          # Plantilla de variables de entorno
├── manage.py                            # Punto de entrada de Django
├── cleanup_media.py                     # Limpieza de archivos temporales
├── docker-compose.yml                  # Orquestacion de contenedores
├── Dockerfile                           # Definicion del contenedor
├── crontab                             # Tareas programadas (cada 5 min)
├── .env                                 # Variables de entorno (API Keys)
├── requirements.txt                     # Dependencias Python
├── translation_generator_app/           # App principal
│   ├── __init__.py
│   ├── admin.py                        # Registro de modelos en admin
│   ├── apps.py                         # Configuracion de la app
│   ├── models.py                       # Modelo translationPost
│   ├── exceptions.py                   # Jerarquia de excepciones
│   ├── urls.py                         # URLs del endpoint /generate-report/
│   ├── services/                       # Logica de negocio
│   │   ├── __init__.py                 # Exporta YouTubeService, TranscriptionService, AnalysisService
│   │   ├── youtube_service.py          # Descarga de video/audio con yt-dlp
│   │   ├── transcription_service.py    # Transcripcion con AssemblyAI
│   │   └── analysis_service.py         # Analisis con Google Gemini (reporte 5 secciones)
│   ├── serializers/                    # Validacion de datos de API
│   │   ├── __init__.py
│   │   └── translation_serializer.py   # TranslationRequestValidator
│   ├── views/                          # Vistas de API
│   │   ├── __init__.py
│   │   └── views_app.py                # ContentAnalysisView (POST /generate-report/)
│   └── __pycache__/
├── ai_translation/                      # Configuracion de Django
│   ├── __init__.py
│   ├── settings.py                     # Configuracion del proyecto
│   ├── urls.py                         # URLs principales del proyecto
│   ├── wsgi.py                         # Punto de entrada WSGI
│   └── asgi.py                         # Punto de entrada ASGI
├── staticfiles/                        # Archivos estaticos recolectados
├── media/                              # Archivos de medios descargados (videos/audio)
└── docs/                               # Documentacion del proyecto
```

├── ARCHITECTURE.md	# Arquitectura tecnica detallada
├── KEY_COMPONENTS_EXPLAINED.md	# Componentes clave y decisiones tecnicas
├── GIT_REORGANIZATION_MANUAL.md	# Manual de reorganizacion de ramas Git
├── PROYECTO_STATE.md	# Estado actual del proyecto
└── venv/	# Entorno virtual Python

Componentes Principales

1. Servicios

Servicio	Responsabilidad	API Externa
YouTubeService	Extraccion de titulo y descarga de audio/video	yt-dlp
TranscriptionService	Transcripcion de audio a texto	AssemblyAI
AnalysisService	Generacion de Reporte de Contenido (5 secciones)	Google Gemini (cadena de 12 modelos con fallback)

AnalysisService es el corazon del proyecto actual. Su metodo `generate_report(transcription_text)` recibe el texto transcrito y genera un reporte estructurado con:

1. **Temas Principales Tratados** - Enumeracion y descripcion de cada tema central
2. **Argumento / Resumen Narrativo** - Narrativa completa del video (inicio, desarrollo, desenlace)
3. **Opiniones o Puntos de Vista Expresados** - Posturas y perspectivas identificadas
4. **Datos o Hechos Clave** - Datos, cifras, nombres, fechas mencionados
5. **Conclusion / Veredicto Final** - Postura general, mensaje final, analisis critico

2. Interfaz Streamlit

La UI principal de la aplicacion. Es un cliente HTTP puro de la API Django (requests). **No** contiene logica de negocio ni importa codigo Django.

- **Cliente HTTP:** Llama a `POST /login/`, `POST /generate-report/` y `POST /logout/` guardando la cookie de sesion. URL del backend con `BACKEND_URL` (default `http://localhost:8000`; `http://backend:8000` en Docker Compose).
- **Seguridad heredada:** Al pasar por la API, el uso via Streamlit queda cubierto por autenticacion y throttling DRF.
- **Gestion de Estado:** Usa `st.session_state` para sesion y resultados
- **Descargas:** Video/audio desde las `video_url/audio_url` de la respuesta; PDF generado localmente con `fpdf2`
- **Manejo de Errores:** Traduce 401/429/400/500 a mensajes amigables

3. API REST

Endpoint disponible:

- **POST /generate-report/** — Genera un Reporte de Contenido

Payload de entrada:

```
{
  "link": "https://youtube.com/watch?v=...",
  "gemini_api_key": "AI..."
}
```

Respuesta:

```
{
  "report": "Reporte de contenido...",
  "title": "Titulo del Video",
  "original_transcription": "Texto original...",
  "transcription_file": "/ruta/al/transcript.txt"
}
```

4. Excepciones Personalizadas

Jerarquia de excepciones para manejo granular de errores:

```
TranslationGeneratorException (Base)
├── YouTubeDownloadException
├── TranscriptionException
├── AnalysisException
└── InvalidDataException
```

5. Base de Datos

Modelo `translationPost`:

Campo	Tipo	Descripcion
youtube_title	CharField(300)	Titulo del video
youtube_link	URLField	URL original de YouTube
generated_content	TextField	Reporte de contenido generado
created_at	DateTimeField(auto_now_add)	Fecha de procesamiento

6. Seguridad: Autenticacion y Throttling

- **Autenticacion:** usuarios nativos de Django. Superusuario con python manage.py createsuperuser; usuarios adicionales en /admin/ → Users → Add user. Streamlit exige login (authenticate()); POST /generate-report/ exige sesion (401 sin login); POST /login/ abre sesion y POST /logout/ la cierra.
- **Throttling DRF:** ScopedRateThrottle por defecto en ai_translation/settings.py. LoginView usa scope login (5/min, anti fuerza bruta); ContentAnalysisView usa scope report (10/hour, operacion costosa). Ajustables con LOGIN_THROTTLE_RATE y REPORT_THROTTLE_RATE. Exceso responde 429.
- **Sesion sin CSRF:** SessionNoCSRF permite usar la cookie de sesion desde clientes no-navegador (curl/scripts).

Cambios Recientes

Aspecto	Antes (Traduccion)	Ahora (Reporte de Contenido)
Proposito	Traducir letras de canciones	Analizar contenido de videos
IA	OpenAI (GPT-4o/Turbo)	Google Gemini (cadena con fallback, default flash-lite)
Endpoint	POST /api/generate-translation/	POST /generate-report/
Output	Traduccion formateada en versos	Reporte de contenido (5 secciones)
Input adicional	target_language	Sin parametro de idioma
Servicio	TranslationService	AnalysisService
Excepcion	TranslationException	AnalysisException
Archivo de servicio	translation_service.py	analysis_service.py
Validacion	openai_api_key + target_language	gemini_api_key

Archivos eliminados: translation_generator_app/services/translation_service.py

Archivos modificados: requirements.txt, .env, app.py, views/views_app.py, serializers/translation_serializer.py, urls.py, views/__init__.py, docs/ARCHITECTURE.md, docs/KEY_COMPONENTS_EXPLAINED.md, README.md

Configuracion del Entorno

El archivo .env contiene las siguientes variables:

```
# API Keys
AAI_API_KEY="..."           # AssemblyAI - para transcripcion (servidor)
# NOTA: la API key de Google Gemini NO va en el .env.
# Cada usuario la ingresa en el sidebar de Streamlit o en el payload REST (gemini_api_key).

# Base de Datos (Neon Cloud PostgreSQL)
DB_NAME=neondb
DB_USER=neondb_owner
DB_PASS=npg_E8PZ4rUfMKQ0
DB_HOST=ep-snowy-queen-adyteach-pooler.c-2.us-east-1.aws.neon.tech
DB_PORT=5432

# Django
SECRET_KEY=django-insecure-...
DEBUG=True
```

Despliegue

La aplicacion esta contenerizada y lista para desplegarse con Docker Compose:

```
# Construir y levantar ambos servicios
docker-compose up --build
```

```
# Solo reconstruir backend tras cambios en requirements.txt
docker compose build --no-cache backend
docker compose up -d backend

# Reiniciar servicios tras cambios en codigo
docker compose restart backend frontend
```

Puertos: - Frontend (Streamlit): <http://localhost:8501> - Backend API (Django): <http://localhost:8000>

Mantenimiento

Limpieza de Archivos Temporales

El script `cleanup_media.py` elimina archivos temporales (.mp4, .mp3, .txt) mayores a 5 minutos del directorio `media/`. Se ejecuta automaticamente cada 5 minutos mediante `crontab`:

```
*/5 * * * * python /backend/cleanup_media.py >> /var/log/cron.log 2>&1
```

Preguntas Frecuentes

- **Que modelo de Gemini se utiliza? Un solo modelo o varios?** Se usan **varios en cadena con fallback automatico**, no uno solo. `PREFERRED_MODELS` tiene 12 modelos en 6 tiers ordenados por cuota restante (snapshot 2026-09-25): empieza en `gemini-3.5-flash-lite` (default, 500 RPD) y, si un modelo responde 429/cuota agotada o 404, `_try_with_fallback()` espera el `retry delay` y prueba el siguiente, hasta `gemma-4-26b/gemma-4-31b` (14.4K RPD) como ultimo recurso. Solo falla si se agotan todos.
 - **Por que no se usa OpenAI?** El proyecto fue migrado de OpenAI a Google Gemini para aprovechar el ecosistema Gemini y la variedad de modelos disponibles con capa gratuita.
 - **Es necesario descargar el video?** Si, se descarga el audio internamente para alimentar a AssemblyAI, que requiere un archivo de audio local. El video y audio se eliminan automaticamente tras 5 minutos mediante el script de limpieza.
 - **Puedo usar la API REST en lugar de Streamlit?** Si, el endpoint `POST /generate-report/` esta disponible. Necesitas enviar la URL de YouTube y tu API Key de Gemini.
 - **Que idiomas soporta el analisis?** El analisis se realiza en español independientemente del idioma de la transcripcion original, ya que el prompt de Gemini instruye generar el reporte exclusivamente en español.
 - **Que ocurre si el video es en otro idioma?** Gemini detecta el idioma implicito del texto y genera el reporte en español. No hay traduccion involucrada, solo analisis y reporte.
 - **Como se manejan los errores?** Existe una jerarquia de excepciones personalizadas (`YouTubeDownloadException`, `TranscriptionException`, `AnalysisException`, `InvalidDataException`) que permite mostrar mensajes especificos al usuario tanto en Streamlit como en la API REST.
 - **Que ocurrio con la reorganizacion de ramas Git?** La rama `main` fue reemplazada por el contenido de `feature/deploy`. La antigua `main` fue renombrada a `main-old` como respaldo, y la rama `develop` fue eliminada. El manual de este proceso se encuentra en `docs/GIT_REORGANIZATION_MANUAL.md`.
-

Checklist de Verificacion

- ☒ `AnalysisService` importa correctamente en contenedor backend
 - ☒ `AnalysisService` importa correctamente en contenedor frontend
 - ☒ `google-generativeai==0.8.4` instalado y funcionando
 - ☒ `GEMINI_API_KEY` provista por el usuario en Streamlit/REST (no va en `.env`)
 - ☒ Endpoint `/generate-report/` disponible (401 sin sesion)
 - ☒ Login/logout por sesion y throttling DRF (429 al exceder tasa)
 - ☒ Streamlit UI actualizada con UI de Reporte de Contenido
 - ☒ `translation_service.py` eliminado
 - ☒ `GIT_REORGANIZATION_MANUAL.md` movido a docs/
 - ☒ Documentacion de arquitectura actualizada
 - ☒ README actualizado con nuevo proposito
-

Historial de Reorganizacion de Ramas Git

Véase `docs/GIT_REORGANIZATION_MANUAL.md` para los detalles de la reorganizacion de ramas del repositorio.